



SDE

Front end application development

ASSIGNMENT1 GOALS

Criteria	What we are looking for
Mockup	Create wireframe from the inputs provided by product manager & relatable inspirations available in public space for product manager to review and make changes before giving signoff for UI development.
UI/ UX	AESTHETIC UI design is of utmost importance and will be key decider. The Designed UI should be responsive, light weight to refresh data and professional & classic in view

Technical Assessment1: Mock-UP and UI

Intro

In next slide the image is layout representation of workshop floors & its workstation(propellia/ sub-Propellia). This workshop floor need to have a digital twin for station wise value monitoring.

TEST:

1. You need to create mockup to represent its layout in best way as per you
2. Covert your mock up to UI page by coding in HTML,CSS, JS to

Additional info:

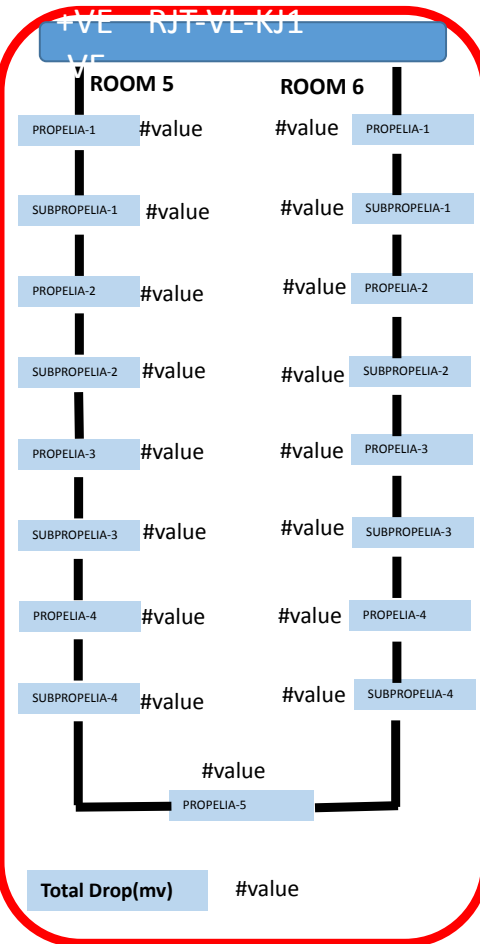
In the layout given in next slide every Line has 2 Rooms(Workshop Floor). Every Workshop has Workstations named Propellia & Subpropellia. Every workstation generates a value which captured by a physical device at the workstation and relayed to the backend. The UI calls the backend to API to provide the most latest value wrt each workstation. In response backend API response is an Json object response. Where the DI Json object 1st level is linename, second level is Roomnumber, third level is workstationname and value pair. The UI thus developed should be consume the response of this API and refresh the data every second automatically.

Submission:

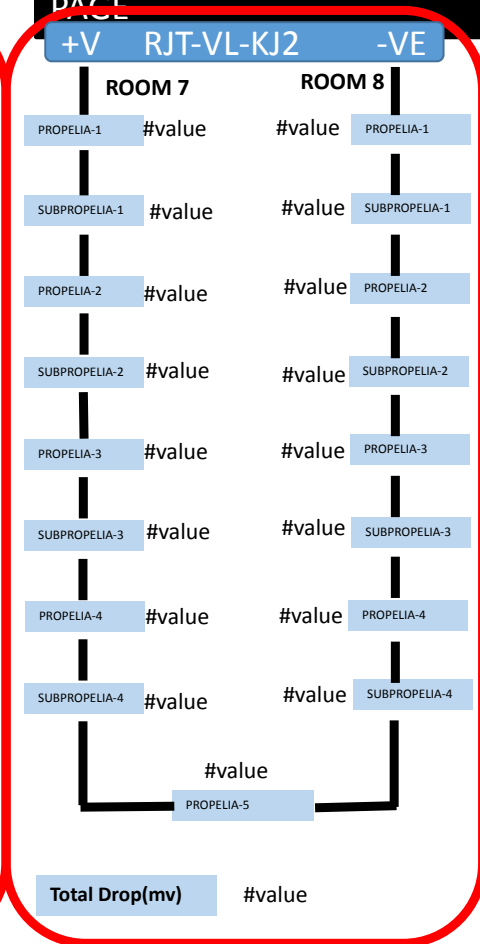
1. Share mockup file
2. Zip of UI ready deployable evaluation.
3. Image of the UI Or link of the live UI
4. API(URI) which the UI consumed

WORKSHOP ROOMWISE / Line wise Value MEASUREMENT

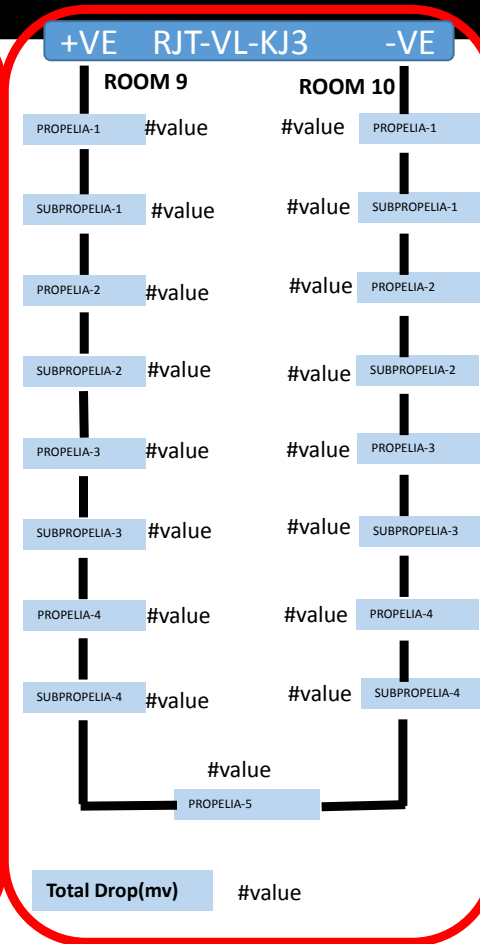
PAGE



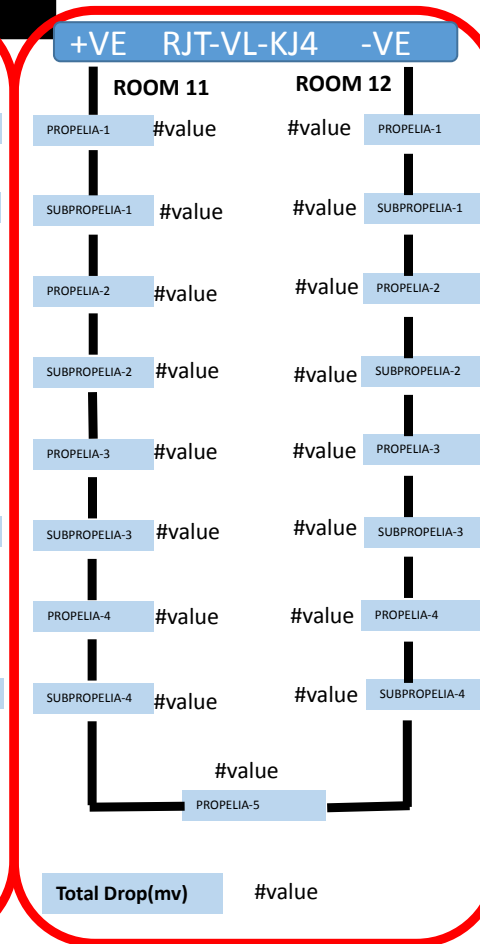
LINE-



LINE-



LINE-



LINE-

ASSIGNMENT2 GOALS

Criteria	What we are looking for
UI/ UX quality	FUCNTIONAL UI design is of utmost importance and will be key decider
Code Quality	Clean, readable, and well-organized code.
Functionality	Does the "Add" and "List" flow work end-to-end?
Efficiency	How you handle the interaction between the FE and BE.
SQL Best Practices	Correct data types and basic query structure.

Technical Assessment: Mini Full-Stack Task Manager (1/3)

Objective: Highly functional in less space UI

application

Develop a functional, single-page "Task Manager" application. The goal is to demonstrate your ability to connect a clean user interface with a server-side API and a relational database.

Features

(1) create new tasks and (2) view an updated list fetched directly from the database. Users can (3) timestamp each entry automatically and (4) modify the task status through a lifecycle of "Created," "WIP," and "Completed." The interface utilizes (5) real-time UI feedback to show state changes without a page reload. It features an "Add Task" button to initialize data and a "Change Status" toggle/button on each individual task to trigger a backend update.

Technical Assessment: Tech stack to be used (2/3)

Project Requirements

1. Front-End (HTML, CSS, JavaScript)

- **UI:** A clean interface to **Add** a new task and **Display** a list of existing tasks.
- **Functionality:** Use JavaScript `fetch()` or `AJAX` to interact with your backend without refreshing the page.
- **Validation:** Basic client-side validation (e.g., preventing empty task names).

2. Back-End (Java or PHP)

- **API Endpoints:** Create two simple RESTful endpoints:
 1. `POST /tasks`: To save a new task to the database.
 2. `GET /tasks`: To retrieve all tasks from the database.
- **Logic:** Handle the incoming request, interact with the DB, and return a JSON response.

3. Database (Postgres)

- Create a single table named `tasks`.
- **Schema:** * `id` (Primary Key, Auto-increment)
 - `task_name` (Varchar)
 - `created_at` (Timestamp)

Technical Assessment: Submissions (3/3)

Architecture Overview

To be drafted by the candidate and submitted as 1 slide. No structuring needed , just dump your architecture idea

Submission Guidelines

To ensure a smooth review process, please provide the following:

- **Source Code:** A link to a Public GitHub or GitLab repository.
- **Live URL:** A link to the hosted application (e.g., using Vercel, Heroku, or a personal server).
- **Database Script:** Include a `.sql` file in the repository to recreate your table structure.
- **README:** A brief file explaining how to run your code locally.



<https://achiredo.com/>



connect@achiredo.com
admin@achiredo.com



+91 93115 67176

THANK YOU!